

Introduction

INF8422 : Perception Robotique et Intelligence Spatiale

Prof. Pierre-Yves Lajoie

**POLYTECHNIQUE
MONTREAL**

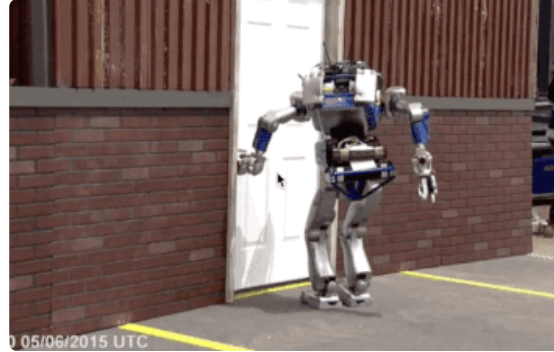
Agenda

1. Logistique du cours
2. Qu'est-ce que la perception robotique et l'intelligence spatiale ?
3. Survol du semestre
4. Robotique autonome
5. Géométrie 3D
6. Capteur LiDAR
7. Iterative Closest Point (ICP)

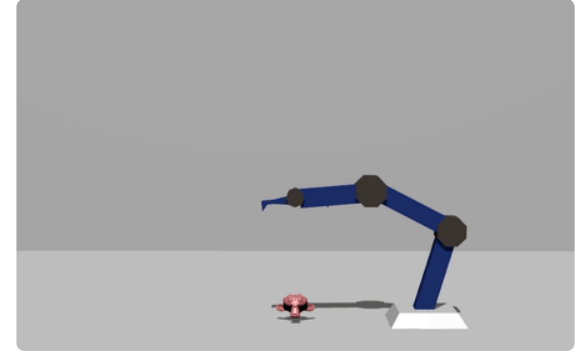
Prérequis



Algèbre linéaire et calcul de base



Programmation en Python



Patience et débrouillardise

Le cours en est à sa première itération.

Aidez-nous à le faire converger : partagez vos commentaires (et vos gradients).

Pierre-Yves Lajoie ing. Ph.D.

Professeur adjoint - Département de génie informatique et génie logiciel

Laboratoire MIST : mistlab.ca



Laboratoire de recherche en robotique autonome

Intelligence Spatiale, Localisation, Cartographie, Planification, Systèmes multi-agent, etc.

Logistique du cours

Évaluations

Évaluation	Pondération
5 Travaux Pratiques	$5 \times 5 \% = 25 \%$
4 meilleurs Quiz / 5	$4 \times 7.5 \% = 30 \%$
Projet de session	25 %
Examen final	20 %

Double seuil: Vous devez avoir au minimum avoir une moyenne de **50% aux évaluations individuelles** (Quizzes et Examen) pour obtenir la note de passage.

Quiz en classe

- Environ aux deux semaines, **25 min**, début de cours.
- Porte sur la matière des cours précédents.
- 5 quiz au total, **seuls les 4 meilleurs** comptent.
- Premier quiz : **11 septembre**.

Travaux Pratiques

Organisation

- **5 TPs** individuels ou en équipe de 2.
- Chargé de laboratoire : **Julien Lagacé**.
- Toutes les ressources sur **Moodle**.
- Rejoignez le **Discord** du cours pour le support (lien sur Moodle).

Labos

Les séances de laboratoire débutent le 9 septembre.

Première remise:

TP 1: mercredi **15 septembre**, 23h30.

Projet de session

Format

- Équipe de **2 à 3 personnes**.
- Rapport de **4 à 6 pages**, double colonne, format **IEEE**.
- Un guide pour le projet est disponible sur **Moodle**.
- Critères: Rigueur scientifique, expérimentations, interprétation des résultats.

Échéancier

9 octobre, 8h30 – Composition de l'équipe + description du sujet (max. 1 page) : objectif (Quoi?), motivation (Pourquoi?) et expérimentations prévues (Comment?).

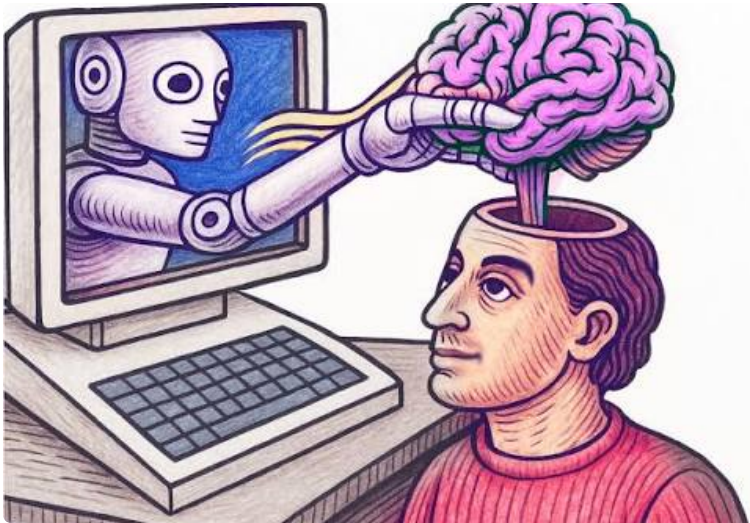
27 novembre – Présentation de **10 minutes**.

22 décembre, 8h30 – Remise du rapport final.

Intelligence artificielle - Point de vue des étudiant

Impact sur vos apprentissages

- Nuit de développer des compétences.
- Baisse de l'esprit critique.
- Créativité limitée.



Vos contraintes

- Vous voulez de bonnes notes.
- Vous n'avez pas beaucoup de temps (cours, recherche, etc.).
- Parfois ce qui est demandé est irréaliste sans l'IA.

Intelligence artificielle - Point de vue de l'enseignant

Problème majeur: Comment concevoir des travaux pratiques permettant de comprendre les concepts vue en classe **ET** suffisamment complexe pour déjouer l'IA...

Solution 1 - Rendre les travaux plus difficiles

Positif:

- Permet des travaux plus ambitieux.
- Plus proche du contexte de travail.
- Apprendre à utiliser les outils d'IA.

Négatif:

- Vous n'avez pas le choix de les utiliser et ça limite les apprentissages.

Solution 2 - Rendre les travaux plus faciles

Proposée dans *Manifeste pour l'éducation humaine*, J. Pilon, 2026

Positif:

- Peut se faire sans l'IA
- Maximise l'apprentissage

Négatif:

- Peut être résoud par l'IA et difficile à évaluer

Intelligence artificielle - Point de vue de l'enseignant

Solution 1 - Projet de session

Projet de session:

Vous pouvez utiliser l'IA si vous voulez et faire un projet le plus ambitieux possible.

Ceci dit, si vous souhaitez *ne pas utiliser l'IA* pour le projet, veuillez nous l'indiquer, on va moduler nos attentes en conséquences.

Solution 2 - Travaux Pratiques

On vous demande de ne pas utiliser l'IA dans les travaux pratiques, pour votre apprentissage.

En contrepartie:

- Support de l'équipe enseignante
- Ajustements si le TP est trop difficile ou trop long

Avant d'utiliser l'IA pour résoudre le TP, venez nous voir et demander de l'aide.

On pourra vous expliquer, vous aider, et ajuster si nécessaire.

Calendrier de la session

Septembre

11 sept. – Quiz 1

15 sept. – TP 1 dû

23-25 sept. – Labo et Cours à distance

29 sept. – TP 2 dû

Octobre

2 oct. – Quiz 2

9 oct. –  Équipe + sujet

20 oct. – TP 3 dû

23 oct. – Quiz 3

Novembre

3 nov. – TP 4 dû

6 nov. – Quiz 4

17 nov. – TP 5 dû

20 nov. – Quiz 5

27 nov. –  Présentation

Décembre

22 déc. –  Rapport dû

Légende

 Quiz

 Remises TP

 Projet

Qu'est-ce que la perception robotique et l'intelligence spatiale ?

Mythe ou Réalité ?

« perception robotique == vision par ordinateur. »

Perception $\neq$ Vision

Partiellement faux. La vision par ordinateur traite des images et vidéos : détection d'objets, segmentation, reconnaissance de formes, etc.

Les robots ont besoin de plus d'informations :

- **Géométrie 3D de tout l'environnement:**
profondeur, obstacles, traversabilité, structures.
- **Propriétés physiques :** articulation, masse, forces.
- **Contraintes temps-réel :** une erreur = une collision.

Vision : « Qu'est-ce que c'est ? »

→ Classification, segmentation, etc.

Perception robotique:

→ Où suis-je?

→ Comment me rendre au point B sans collision?

→ Comment prendre un objet?

→ Quel sera l'état de l'environnement suivant une action?

Mythe ou Réalité ?

« La perception robotique est plus difficile que la vision par ordinateur. »

Tout est relatif

Parfois oui :

- Doit produire des sorties plus riches (3D, propriétés physiques, etc).
- Sensible à la vitesse de traitement.
- Les erreurs ont des conséquences réelles.

Parfois non : le robot peut agir pour simplifier la perception.

Le robot n'est pas passif

Contrairement à la vision classique, le robot peut :

- Se déplacer pour trouver un meilleur angle de vue.
- S'approcher pour améliorer la résolution.
- Interagir pour révéler des propriétés cachées.

Mythe ou Réalité ?

« La perception robotique se limite aux données visuelles. »

La perception robotique est multimodale

Faux. Chaque modalité apporte une information complémentaire.

Vision

Caméra RGB, RGB-D, stéréo

Géométrie active

LiDAR, Radar, Sonar

Inertiel

IMU (accéléromètre, gyroscope, magnétomètre)

Tactile

Force, pression, texture

Infrastructure

GPS, UWB, WiFi

Acoustique

Microphones

Mythe ou Réalité ?

« La perception robotique == l'estimation d'état. »

Pas seulement.

La perception robotique cherche à construire une représentation de l'environnement pour la planification/autonomie.

- Certaines propriétés sont qualitatives (ex. sémantique).
- Certaines représentations sont implicites (ex. espace latent d'un réseau neuronal).
- L'estimation d'état est généralement passive, alors que la perception robotique peut être active.
- La structure de l'environnement et ses ambiguïtés pour informer l'estimation de l'incertitude.



Survol du semestre

Le cours couvre plusieurs sujets :

1. **Fondements:** Géométrie 3D et outils mathématiques
2. **SLAM (Cartographie et Localisation Simultanées):** Extraction de données de capteurs, Estimation d'état.
3. **Représentation :** Comment modéliser le monde et gérer l'incertitude.
4. **Perception Dynamique et en Action :** Dynamique, interaction, manipulation, multi-agent.
5. **Applications :** Sujets avancés et études de cas.



Robotique autonome

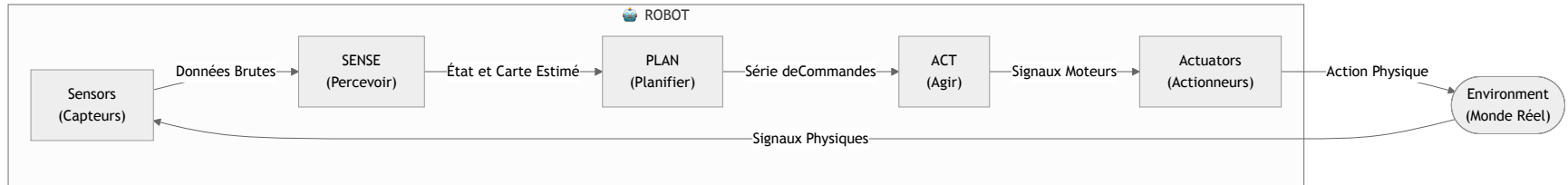
Le cycle de l'autonomie : Percevoir-Planifier-Agir

Le paradigme classique de la robotique autonome (Sense-Plan-Act) :

- **Percevoir (Sense)** : Acquisition de données et estimation de l'état du monde.
- **Prédire (Predict)** : Prédire l'évolution de l'environnement.
- **Planifier (Plan)** : Prise de décision basée sur l'état estimé pour atteindre un but.
- **Agir (Act)** : Exécution des commandes motrices. Change l'état du robot et de l'environnement.

Rôle de la perception

C'est l'interface entre le monde physique réel et le modèle numérique du robot et de son environnement.



Paradigme 1 : La Perception pour l'Autonomie

Historiquement, la perception est vue comme étant au service de la navigation.

- **Objectif** : Permettre au robot de se déplacer sans collision et de se localiser.
- **Approche classique** : Le robot crée et maintient une carte de son environnement en temps réel. Le défi est de trouver la carte et l'estimation de position qui expliquent le mieux les données bruitées des capteurs.

Applications types

- Véhicules autonomes (Waymo, Tesla).
- Robots aspirateurs (iRobot).
- Logistique (Gestion d'entrepôt, Amazon).

Paradigme 2 : L'Autonomie pour la Perception

Perception Active : Le robot agit pour mieux percevoir.

- **Exemple** : "Se déplacer pour voir derrière l'obstacle" ou "S'approcher pour mieux classifier".
- **Objectif** : Maximisation de l'information, qualité de la carte, etc.

Applications types

- Inspection d'infrastructures.
- Recherche et sauvetage.
- Exploration d'environnements inconnus.
- Agriculture de précision.

Géométrie 3D

Géométrie 3D

La navigation robotique repose sur la géométrie 3D :

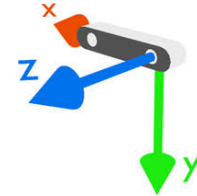
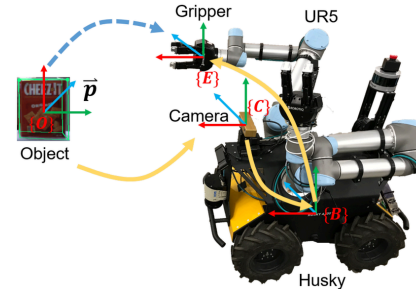
- **Repères de coordonnées** (Frames)
- **Positions et Translations**
- **Représentation de l'Orientation**
- **Représentation de la Pose** (Position + Orientation)

Repères de Coordonnées (Coordinate Frames)

Un repère est un ensemble d'axes orthogonaux attachés à un corps.

Repères usuels en robotique :

- **Robot frame (r)** : Attaché au robot (souvent au centre de masse).
- **Sensor frame ($c, l \dots$)** : Attaché au capteur (ex : caméra, LiDAR).
- **World frame (w)** : Fixe dans l'environnement (ex : point de départ).

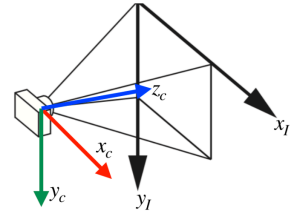
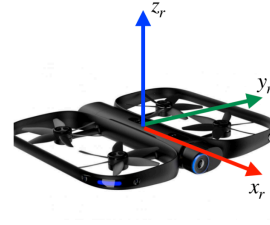


Convention Main Droite (Right-Handed)

$$\mathbf{x} \times \mathbf{y} = \mathbf{z}$$

Mnémoniques

- Pouce (z), Index (x), Majeur (y).
- Pouce vers z , les doigts s'enroulent de x vers y .



Repères usuels (gauche: robot frame, droite: camera/image frames)

Conventions de Repères

Robot Frame (r)

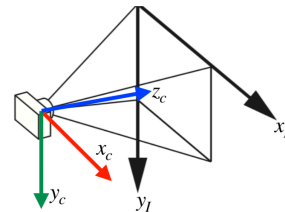
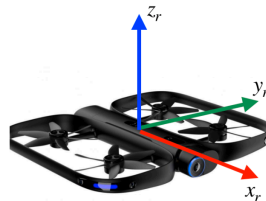
- Origine : Centre de masse.
- x_r : Vers l'avant (Forward).
- y_r : Vers la gauche (Left).
- z_r : Vers le haut (Up).

Image Frame (i) (2D)

- Origine : Coin haut-gauche.
- x droite, y bas.

Camera Frame (c)

- Origine : Centre optique.
- z_c : Regarde la scène (Optical Axis).
- x_c : Vers la droite.
- y_c : Vers le bas.

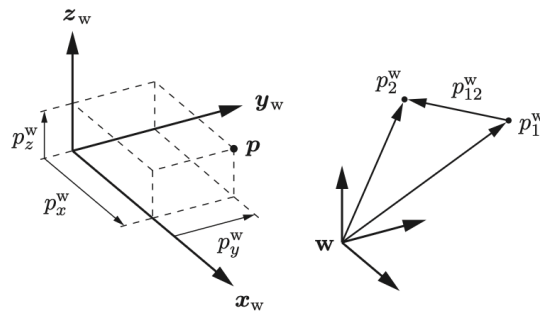


Points et Positions

La position d'un point p dans le repère w est un vecteur :

$$p^w = \begin{bmatrix} p_x^w \\ p_y^w \\ p_z^w \end{bmatrix} \in \mathbb{R}^3$$

Les scalaires p_x^w, p_y^w, p_z^w sont les projections sur les axes x_w, y_w, z_w .



Translations et Composition

Déplacement (Translation) : entre deux points p_1 et p_2 :

$$p_{12}^w = p_2^w - p_1^w$$

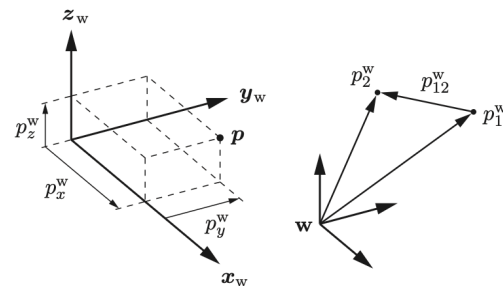
Composition :

$$p_2^w = p_1^w + p_{12}^w$$

Inverse :

$$p_{12}^w = -p_{21}^w$$

Note : La notation explicite le repère (w , i.e. world) en exposant.



Représentations de la Rotation

Comment décrire l'orientation du repère r (robot) par rapport à w (monde) ?

1. **Angles d'Euler** (Roll-Pitch-Yaw)
2. **Axe-Angle** (Axis-Angle)
3. **Quaternions**
4. **Matrice de Rotation** (Rotation Matrix)

1. Angles d'Euler (Roll-Pitch-Yaw)

Une rotation RPY exige de préciser l'ordre des axes.

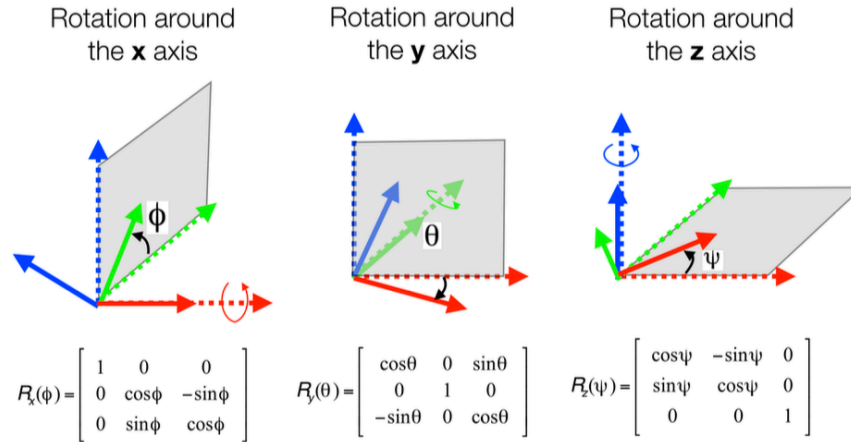
**Convention extrinsèque XYZ
(axes fixes) :**

$$R = R_z(\gamma) R_y(\beta) R_x(\alpha)$$

α : roll autour de x ; β : pitch
autour de y ; γ : yaw autour de z .

Avantages

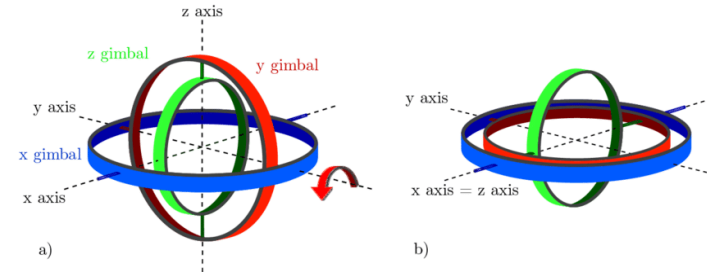
Intuitif. Minimal (3 paramètres).



Angles d'Euler – Limitations

Gimbal Lock (Singularité)

Si le pitch $\beta = \pm\pi/2$, on perd un degré de liberté : deux axes de rotation deviennent colinéaires.



Autres inconvénients :

- Calculs trigonométriques coûteux pour la composition et l'inverse.
- Plusieurs conventions possibles (RPY, YPR, XYZ, ZYX...) → sources de confusion fréquentes.

2. Représentation Axe-Angle

Théorème d'Euler : toute rotation est équivalente à une rotation d'angle θ autour d'un axe fixe $\mathbf{u}$ ($\|\mathbf{u}\| = 1$).

Le **vecteur de rotation** $\boldsymbol{\omega} = \theta \mathbf{u} \in \mathbb{R}^3$ contient les trois paramètres indépendants.

Formule de Rodrigues (Axe-Angle $\rightarrow$ Matrice) :

$$R_r^w = \cos(\theta) \mathbf{I}_3 + \sin(\theta) [\mathbf{u}]_{\times} + (1 - \cos(\theta)) \mathbf{u} \mathbf{u}^T$$

Où $[\mathbf{u}]_{\times}$ est la matrice antisymétrique (*skew-symmetric*) telle que $[\mathbf{u}]_{\times} \mathbf{v} = \mathbf{u} \times \mathbf{v}$:

$$[\mathbf{u}]_{\times} = \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix}$$

Conversion Matrice $\rightarrow$ Axe-Angle

Comment retrouver $(\mathbf{u}, \theta)$ à partir d'une matrice R ?

Angle θ : via la trace de la matrice :

$$\text{tr}(R) = 1 + 2 \cos(\theta) \implies \theta = \arccos\left(\frac{\text{tr}(R) - 1}{2}\right)$$

Axe $\mathbf{u}$: vecteur propre associé à la valeur propre $\lambda = 1$:

$$R \mathbf{u} = \mathbf{u}$$

L'axe de rotation est invariant par la rotation.

Valeurs propres de R

Les valeurs propres de toute matrice de rotation sont :

$$\{1, e^{i\theta}, e^{-i\theta}\}$$

3. Quaternions

Extension des nombres complexes : $q = q_4 + i q_1 + j q_2 + k q_3$ avec $i^2 = j^2 = k^2 = ijk = -1$, $ij = -ji = k$, $jk = -kj = i$, $ki = -ik = j$.

Convention : $q = \begin{bmatrix} v \\ w \end{bmatrix} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix},$

Lien avec Axe-Angle ($\mathbf{u}, \theta$) :

$$q = \begin{bmatrix} \mathbf{u} \sin(\theta/2) \\ \cos(\theta/2) \end{bmatrix}$$

avec v la partie vectorielle et w la partie scalaire.

Lien avec la matrice de rotation $R(q)$:

$$R(q) = \begin{bmatrix} q_1^2 - q_2^2 - q_3^2 + q_4^2 & 2(q_1 q_2 - q_3 q_4) & 2(q_1 q_3 + q_2 q_4) \\ 2(q_1 q_2 + q_3 q_4) & -q_1^2 + q_2^2 - q_3^2 + q_4^2 & 2(q_2 q_3 - q_1 q_4) \\ 2(q_1 q_3 - q_2 q_4) & 2(q_2 q_3 + q_1 q_4) & -q_1^2 - q_2^2 + q_3^2 + q_4^2 \end{bmatrix}$$

Opérations sur les Quaternions Unitaires

Pour représenter une rotation : quaternions unitaires ($\|q\| = 1$), avec $q = \begin{bmatrix} v \\ w \end{bmatrix}$.

Composition (Produit) :

$$q_a \otimes q_b = \begin{bmatrix} w_a v_b + w_b v_a + v_a \times v_b \\ w_a w_b - v_a^T v_b \end{bmatrix}$$

Expression d'un point dans un repère différent :

$$q_r^w \otimes \begin{bmatrix} p^r \\ 0 \end{bmatrix} \otimes (q_r^w)^{-1} = \begin{bmatrix} p^w \\ 0 \end{bmatrix} = \begin{bmatrix} R(q_r^w) p^r \\ 0 \end{bmatrix}$$

Inverse :

$$q^{-1} = \begin{bmatrix} -q_{1:3} \\ q_4 \end{bmatrix}$$

Avantages des Quaternions

Avantages

- **Sans singularité** – pas de Gimbal Lock.
- **Compact** – 4 scalaires et une contrainte de normalisation, donc 3 degrés de liberté.
- **Calculs efficaces** – la composition ne requiert pas de trigonométrie : 16 multiplications vs 27 pour les matrices.
- **Génération** – il suffit de générer 4 nombre réel et de normaliser pour obtenir un quaternion valide.

Inconvénient : Double Couverture

q et $-q$ représentent la **même rotation**.

Cela cause des problèmes pour les interpolations et l'optimisation (discontinuités).

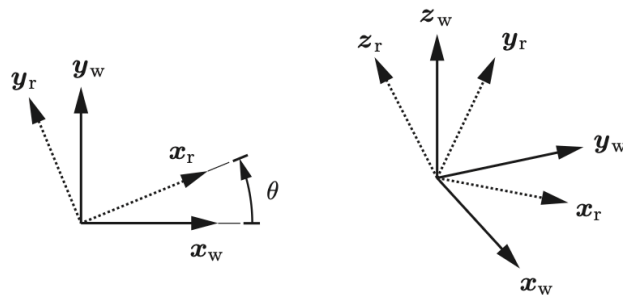
4. Matrice de Rotation

On projette les axes de r (x_r, y_r, z_r) dans le repère w et on empile ces vecteurs en colonnes.

$$R_r^w = \begin{bmatrix} | & | & | \\ x_r^w & y_r^w & z_r^w \\ | & | & | \end{bmatrix} \in SO(3)$$

Exemple 2D :

$$R_r^w = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \in SO(2)$$



Propriétés du Groupe $SO(3)$

Orthogonalité

Colonnes unitaires et perpendiculaires :

$$(R_r^w)^T R_r^w = I_3 \implies (R_r^w)^{-1} = (R_r^w)^T$$

Repère direct (Right-Handed)

$$\det(R_r^w) = +1$$

Le produit mixte des colonnes : $(x \times y) \cdot z = 1$.

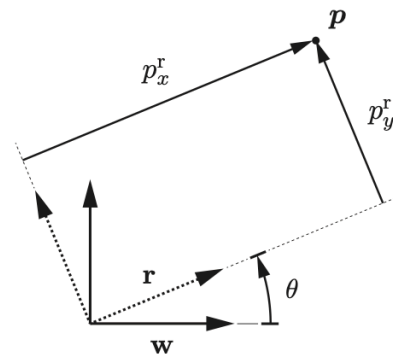
Opérations avec Matrices de Rotation

Rotation de point : convertir p^r (repère robot) en p^w (repère monde) :

$$p^w = R_r^w p^r$$

Composition : rotation du capteur dans le repère du monde :

$$R_c^w = R_r^w R_c^r$$



Non commutatif

$$R_r^w R_c^r \neq R_c^r R_r^w$$

Inverse :

$$R_r^w = (R_w^r)^{-1} = (R_w^r)^T$$

Poses et Transformations Rigides

Pose et Transformation Rigide

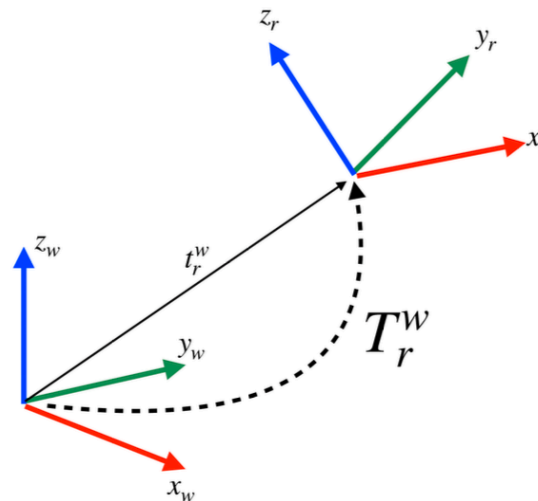
Une **Pose** combine position et orientation de r relative à w :

$$(R_r^w, t_r^w)$$

- R_r^w : Orientation du robot r dans le repère monde w .
- t_r^w : Position du repère du robot r dans le repère monde w .

Transformation de point (changement de repère) :

$$p^w = R_r^w p^r + t_r^w$$



Coordonnées Homogènes

Les coordonnées homogènes permettent de représenter une transformation affine par une multiplication matricielle 4×4 .

Point homogène : $\tilde{p}^r = \begin{bmatrix} p^r \\ 1 \end{bmatrix}$

Matrice de Transformation $T_r^w \in SE(3)$:

$$T_r^w = \begin{bmatrix} R_r^w & t_r^w \\ 0_{1 \times 3} & 1 \end{bmatrix}$$

L'équation de changement de repère devient :

$$p^w = R_r^w p^r + t_r^w \quad \Longleftrightarrow \quad \tilde{p}^w = T_r^w \tilde{p}^r$$

Opérations sur les Poses $SE(3)$

Transformation d'un point :

$$\begin{bmatrix} p^w \\ 1 \end{bmatrix} = \begin{bmatrix} R_r^w & t_r^w \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p^r \\ 1 \end{bmatrix}$$

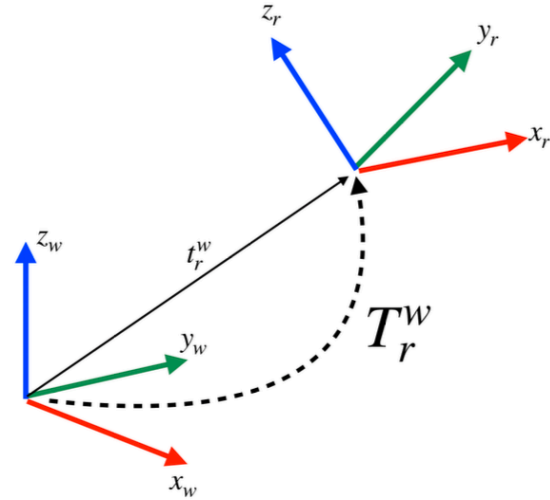
Composition (repère intermédiaire commun) :

$$T_c^w = T_r^w T_c^r$$

Inverse :

$$T_w^r = (T_r^w)^{-1} = \begin{bmatrix} (R_r^w)^T & -(R_r^w)^T t_r^w \\ 0 & 1 \end{bmatrix}$$

Note : $(T_r^w)^{-1} \neq (T_r^w)^T$ car la matrice T n'est pas orthogonale.



Résumé des Représentations

Représentation	Scalars (constraints)	Singularities	Usage principal
Matrice de Rotation	9 (6)	Non	Opérations sur les vecteurs
Angles d'Euler (RPY)	3	Oui	Affichage intuitif
Axe-Angle	3 (ω)	Oui, à $\theta = 0$	Calculs, conversions
Quaternion	4 (1)	Non	Stockage, génération

En général pour

- **Stocker / générer** → Quaternion
- **Calculer / appliquer** → Matrice de Rotation ou Axe-Angle
- **Communiquer à un humain** → Angles d'Euler

Capteur LiDAR

Taxonomie des Capteurs

Proprioceptifs (Interne)

Mesurent l'état interne du système.

- Ex : Encodeurs de roues, IMU, Batterie.

Passif

Reçoit l'énergie. Consomme moins d'énergie.

- Ex: Caméras

Extéroceptifs (Externe)

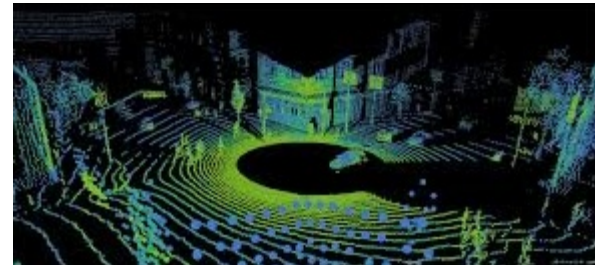
Mesurent l'environnement.

- Ex : Caméras, LiDAR, Radar.

Actif

Émet l'énergie. Consomme beaucoup d'énergie. Généralement plus robuste.

- Ex: LiDAR, Sonar, Radar



Capteurs Time-of-Flight (ToF)

Principe de base pour LiDAR, Radar, Sonar.

$$d = \frac{c \cdot \Delta t}{2}$$

- c : Vitesse de l'onde ($3 \cdot 10^8$ m/s pour la lumière, **343** m/s pour le son).
- Δt : Temps aller-retour.

Comparaison

- **LiDAR** : Faisceau collimaté (Laser). Mesure ponctuelle très précise.
- **Sonar** : Cône d'émission. Mesure la première réflexion dans le cône (ambiguïté angulaire).

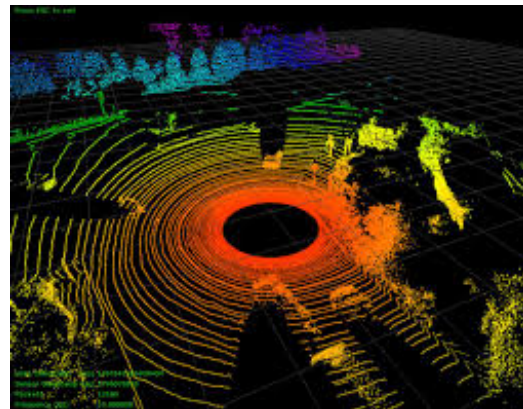
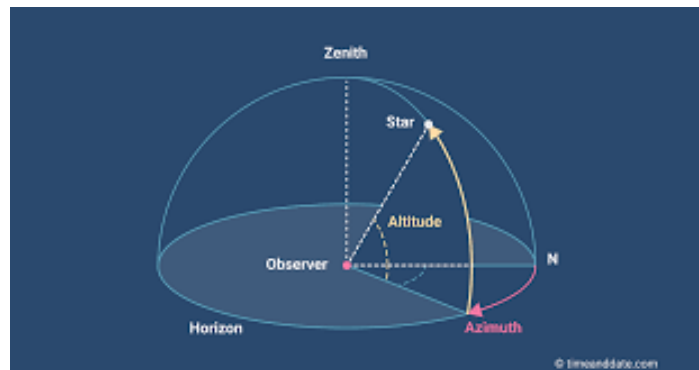
LiDAR 3D et Nuages de points

Combinaison de distance ToF + mécanisme de scan (miroir rotatif).

- **Output** : Nuage de points $\mathcal{P} = \{p_1, \dots, p_N\}$ avec $p_i = (x, y, z, \text{reflectivity})$.
- **Géométrie sphérique** :

$$p_i = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix} = r \begin{bmatrix} \cos \phi \sin \theta \\ \cos \phi \cos \theta \\ \sin \phi \end{bmatrix}$$

où r est la distance, θ l'azimut (rotation), ϕ l'élévation (laser ID).



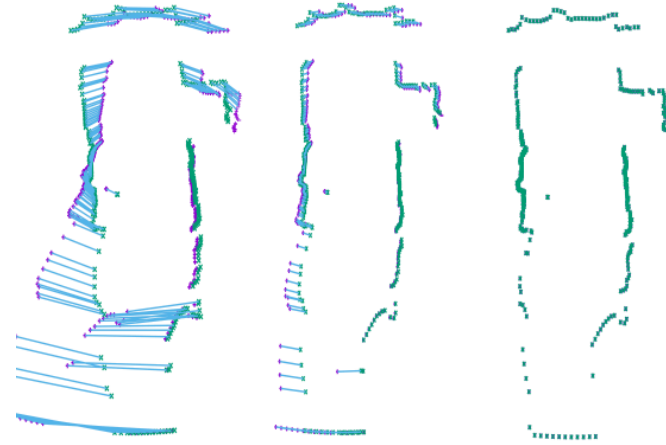
Un problème du LiDAR : Distorsion de mouvement

Un scan (rotation du laser) n'est pas instantané (ex : 10 Hz $\rightarrow$ 100 ms).

Le Phénomène

Le point p_0 est acquis à t . Le point p_N est acquis à $t + 100$ ms.

- Entre-temps, le robot peut avoir avancé de 2 m (vitesse de 72 km/h).
- Le robot peut avoir fait une rotation rapide ou subi des vibrations.
- Résultat : Les murs droits deviennent courbes, les objets sont dédoublés.



Gauche: scan distordu. Milieu: correction. Droite: scan corrigé (Deskew).

Deskewing – Correction de la distorsion de mouvement

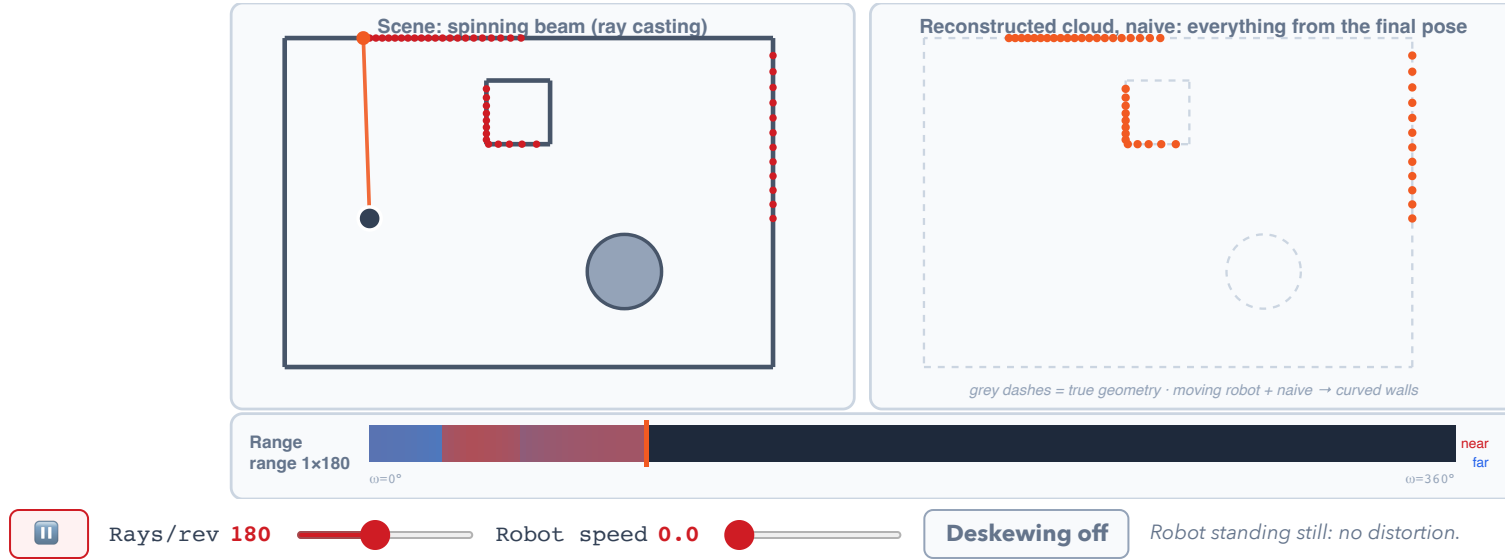
La Solution : Deskewing

On projette tous les points à un temps de référence t_{ref} (début ou fin de scan).

$$p_{\text{corrigé}} = T_{\text{ref}}^{-1} \cdot T(t_i) \cdot p_{\text{brut}}$$

Nécessite de connaître la vitesse du robot à haute fréquence (IMU) pour interpoler $T(t_i)$.

A scan is not a snapshot

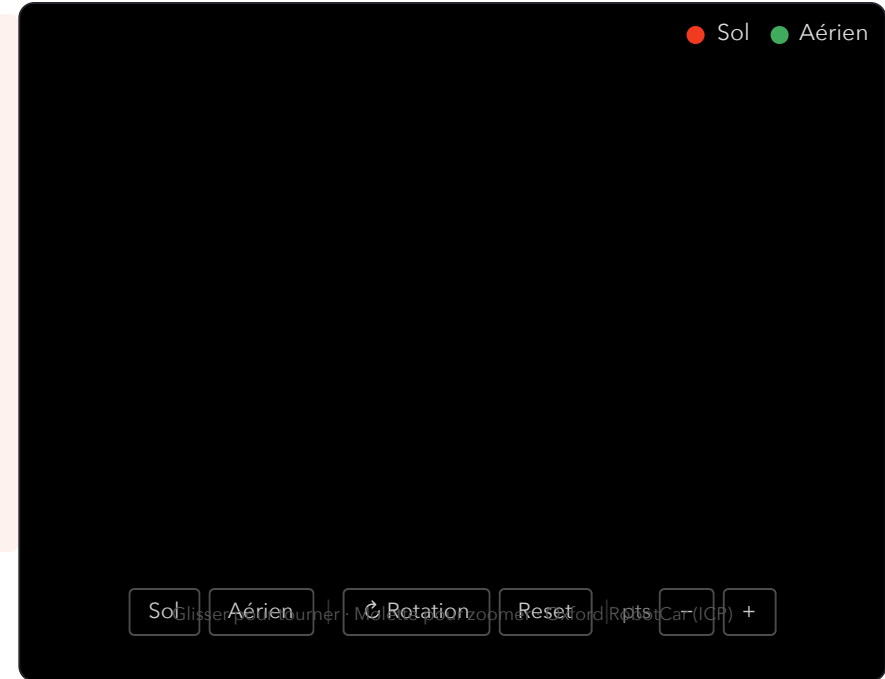


Iterative Closest Point (ICP)

Alignement de Nuages de Points

Algorithme fondamental pour la robotique autonome.

- Estimer le déplacement d'une voiture autonome.
- Associer un nuage de points avec un modèle CAD.
- Guider un outil chirurgical
- etc.



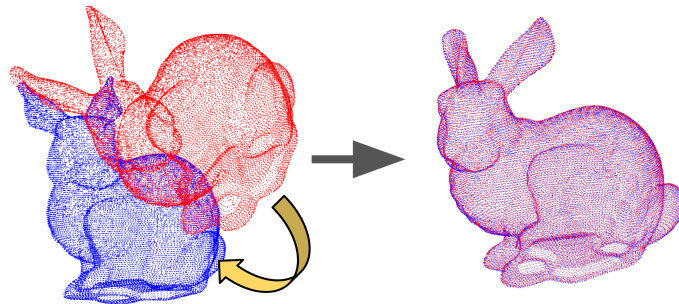
Alignement de Nuages de Points

Le Problème : Soit deux nuages de points :

- **Cible (Target, Q) :** Le nuage fixe (ex : la carte).
- **Source (Source, P) :** Le nuage à aligner (ex : scan courant).

Objectif : Trouver la transformation rigide (R, t) qui aligne P sur Q en minimisant l'erreur quadratique moyenne.

$$E(R, t) = \sum_{i=1}^N \|q_i - (Rp_i + t)\|^2$$



L'œuf ou la poule ?

Le dilemme

- Si on connaissait les **correspondances** exactes, on pourrait calculer (R, t) directement en une seule étape (solution fermée).
- Si on connaissait la **transformation** (R, t) , on pourrait trouver les correspondances facilement.

Solution de l'ICP : Procéder itérativement.

1. Estimer les correspondances (au plus proche voisin).
2. Calculer la transformation optimale pour ces correspondances.
3. Appliquer la transformation et recommencer.

L'Algorithme ICP Standard

Entrée : P (Source), Q (Cible), estimation initiale (R_0, t_0) .

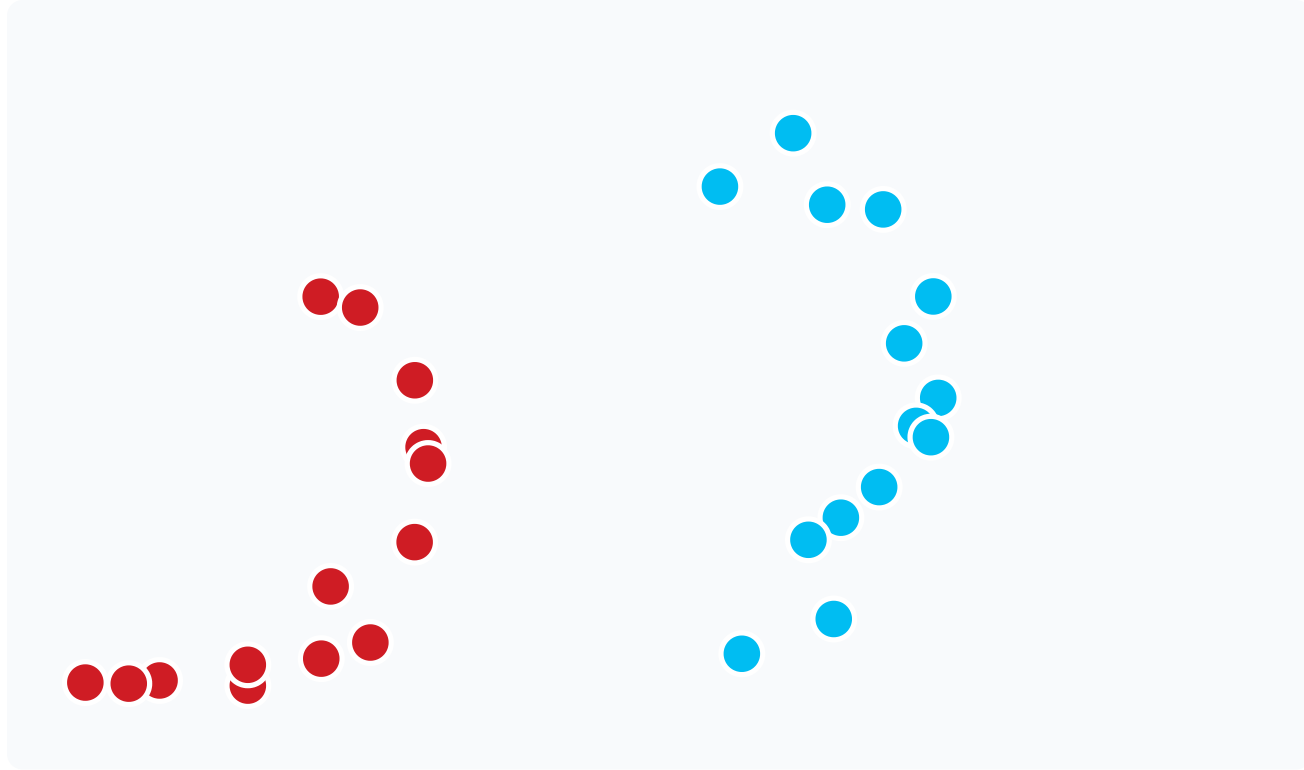
1. **Association de données :** Pour chaque point $p_i \in P$, trouver le point le plus proche $q_j \in Q$ (Nearest Neighbor).
2. **Estimation :** Trouver (R, t) qui minimisent l'erreur pour ces paires :

$$(R^*, t^*) = \operatorname{argmin}_{R, t} \sum \|q_j - (Rp_i + t)\|^2$$

3. **Transformation :** Appliquer $P \leftarrow R^*P + t^*$.
4. **Convergence :** Si le changement d'erreur $< \epsilon$, arrêter. Sinon retour à 1.

Note : L'étape 1 (Nearest Neighbor) est coûteuse $\rightarrow$ structures KD-Tree.

Démonstration interactive : ICP



● Cible Q (fixe) ● Source P (mobile)

▶ Démarrer

▶ Auto

↺ Reset

Le Problème du Voisin le Plus Proche

Pour chaque point $p_i \in P$, trouver le point le plus proche $q_j \in Q$.

Solution naïve :

- Pour chacun des n points de P : parcourir les n points de Q et calculer la distance.
- Trouver une correspondance : $O(n)$
- Trouver **toutes** les correspondances : $O(n^2)$ ← trop lent !

Problème de passage à l'échelle

Un LiDAR produit ~100 000 points par scan. $O(n^2)$ signifie 10^{10} opérations par itération ICP – impraticable en temps réel.

K-D Tree : Structure de données spatiale

Solution efficace : organiser les points dans un **arbre binaire de partitionnement spatial**.

Principe :

- Construire un arbre binaire représentant le nuage cible Q .
- Parcourir l'arbre au lieu d'une liste pour trouver les correspondances.

K-D Tree : Construction de l'arbre

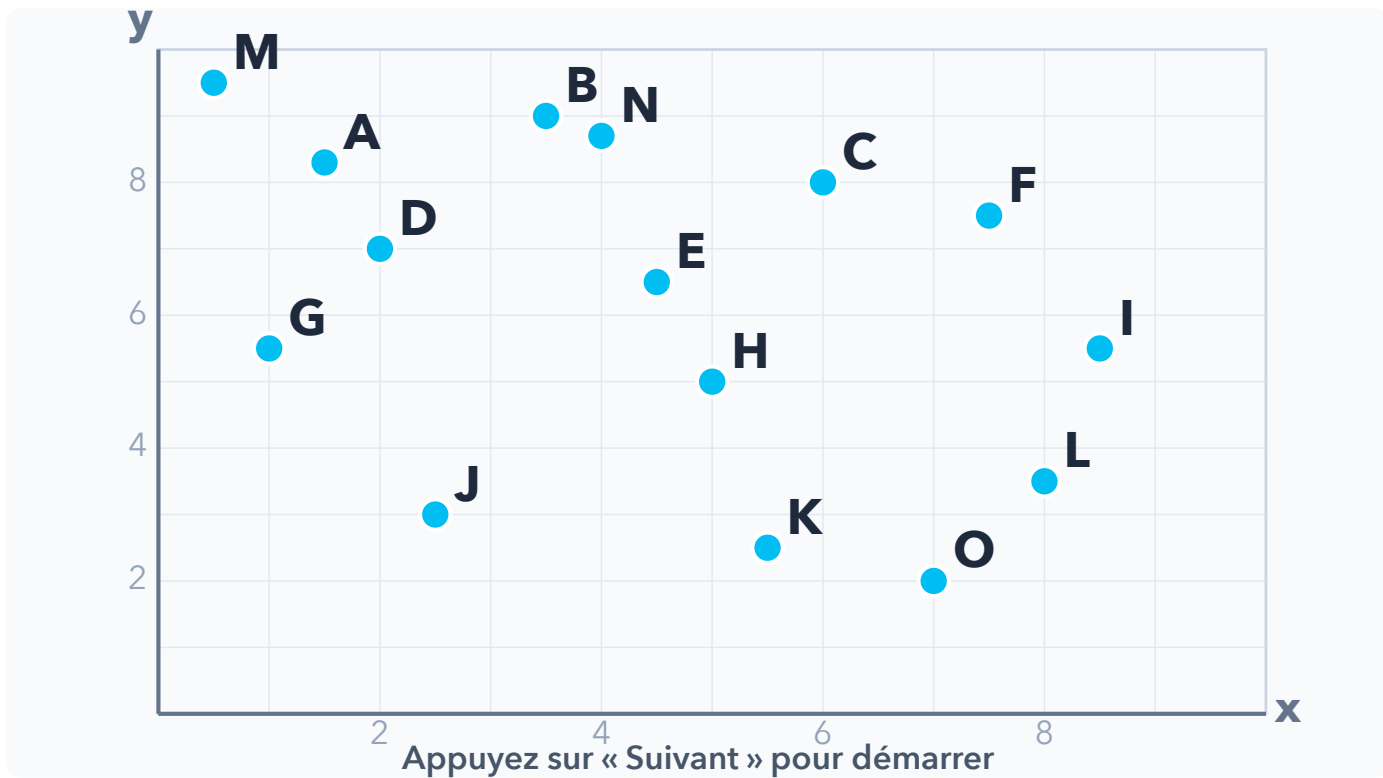
Étapes récursives :

1. Choisir un axe (en alternance $x, y, z, \dots$).
2. Diviser le nuage en deux en fonction du **point médian** sur cet axe.
3. Ajouter le point médian à l'arbre (nœud).
4. Aller récursivement dans les deux moitiés.

Niveau 0 (axe X) : médiane → nœud racine

```
├─ Niveau 1 (axe Y) : médiane gauche
│   └─ Niveau 2 (axe X) : ...
│       └─ Niveau 2 (axe X) : ...
└─ Niveau 1 (axe Y) : médiane droite
    └─ ...
```

Démonstration interactive : Construction du K-D Tree



Prof. 0 Prof. 1 Prof. 2 Prof. 3 Prof. 4

◀ Précédent

Suivant ▶

▶ Auto

↺ Reset

K-D Tree : Complexité de la construction

Pourquoi $O(n \log n)$?

L'arbre a $\log_2 n$ niveaux de profondeur. À chaque niveau, on partitionne **tous les n points** pour trouver la médiane :

$$\underbrace{\log n}_{\text{niveaux}} \times \underbrace{O(n)}_{\text{médiane}} = O(n \log n)$$

Intuition : coupes récursives

n points $\rightarrow$ 1 médiane (niveau 0)
 $n/2 + n/2 \rightarrow$ 2 médianes (niveau 1)
...
1 point par feuille (niveau $\log_2 n$)

À chaque niveau, le travail total est $O(n)$. La somme sur $\log n$ niveaux donne $O(n \log n)$.

K-D Tree : Recherche du plus proche voisin

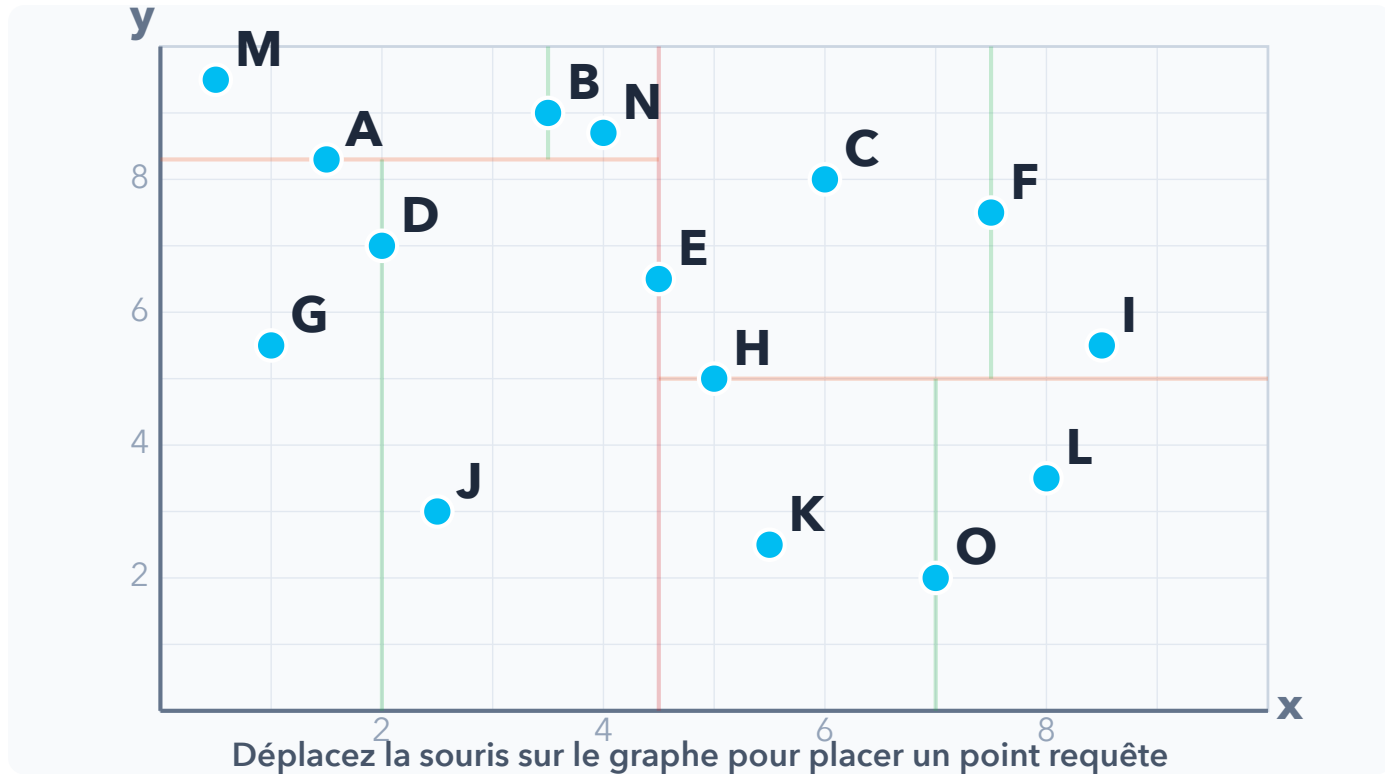
Algorithme de recherche pour un point requête q :

1. À partir de la **racine**, calculer la distance avec le nœud courant.
2. Comparer q selon l'axe du nœud → descendre dans la branche correspondante.
3. À la feuille, conserver le meilleur candidat trouvé.
4. **Remonter l'arbre** pour vérifier s'il peut exister un point plus proche dans l'autre branche (comparer la distance au plan de séparation).

Étape clé : la vérification en remontant

Une sphère de rayon = distance courante peut traverser le plan de séparation → il faut explorer l'autre sous-arbre si c'est le cas.

Démonstration interactive : Recherche dans le K-D Tree



K-D Tree : Complexité de la recherche

Pourquoi $O(\log n)$ en moyenne ?

Chaque comparaison sur un axe **élimine $\sim \frac{1}{2}$ de l'espace restant**, comme une recherche binaire :

$$\underbrace{\log n}_{\text{décisions}} \times O(1) = O(\log n)$$

Opération	Complexité
1 correspondance	$O(\log n)$ moy.
Pire cas (dim. élevée)	$O(n)$
n correspondances	$O(n \log n)$ moy.

Pourquoi $O(n)$ en pire cas ?

En remontant l'arbre, on vérifie si la sphère de rayon = distance courante **traverse le plan de séparation**. En haute dimension ($k \gg 3$), cette sphère traverse presque tous les hyperplans $\rightarrow$ on finit par visiter tous les nœuds.

Gain vs solution naïve

$$O(n^2) \xrightarrow{\text{K-D Tree}} O(n \log n)$$

Pour $n = 100\,000$ points :

- Naïf : 10^{10} opérations
- K-D Tree : $\sim 1,7 \times 10^6$ opérations

K-D Tree : Limitations

Avantages :

- Très efficace pour représenter des **points** en faible dimension.
- Simple à implémenter, largement disponible (Open3D, PCL, scipy).

Limitations :

- Pour des géométries complexes (surfaces, volumes), on préfère les **hiérarchies de volumes englobants** (BVH).
- L'efficacité **diminue avec la dimension k** – en haute dimension, on doit remonter l'arbre très souvent (*malédiction de la dimensionnalité*).

Approximation :

Peut être construit en $O(n \log n)$ en utilisant un algorithme **approximatif** en $O(n)$ pour trouver les médianes (algorithme de sélection linéaire).
Nécessite de rééquilibrer l'arbre périodiquement si les données changent.

En pratique

Les bibliothèques comme **FLANN** ou **nanoflann** utilisent des variantes approximatives (ϵ -NN) pour accélérer encore la recherche.

L'Algorithme ICP Standard

Entrée : P (Source), Q (Cible), estimation initiale (R_0, t_0) .

1. **Association de données :** Pour chaque point $p_i \in P$, trouver le point le plus proche $q_j \in Q$ (Nearest Neighbor).
2. **Estimation :** Trouver (R, t) qui minimisent l'erreur pour ces paires :

$$(R^*, t^*) = \operatorname{argmin}_{R, t} \sum \|q_j - (Rp_i + t)\|^2$$

3. **Transformation :** Appliquer $P \leftarrow R^*P + t^*$.
4. **Convergence :** Si le changement d'erreur $< \epsilon$, arrêter. Sinon retour à 1.

Note : L'étape 1 (Nearest Neighbor) est coûteuse $\rightarrow$ structures KD-Tree.

Résolution par SVD : Centrage

Comment résoudre l'étape 2 de manière exacte ? (Méthode de Kabsch / Arun).

1. Calcul des centres de masse (Centroids) :

$$\mu_P = \frac{1}{N} \sum_{i=1}^N p_i, \quad \mu_Q = \frac{1}{N} \sum_{i=1}^N q_i$$

2. Centrage des nuages : On définit les points par rapport au centre du nuage :

$$p'_i = p_i - \mu_P, \quad q'_i = q_i - \mu_Q$$

Intuition : Cela permet de découpler la rotation de la translation.

Résolution par SVD : Rotation

3. Matrice de covariance croisée (Cross-Covariance Matrix) :

$$W = \sum_{i=1}^N q'_i (p'_i)^T = Q' P'^T$$

4. Décomposition en Valeurs Singulières (SVD) :

$$W = U D V^T$$

5. Calcul de la Rotation :

$$R = U V^T$$

Intuition : Pourquoi la SVD donne la rotation ?

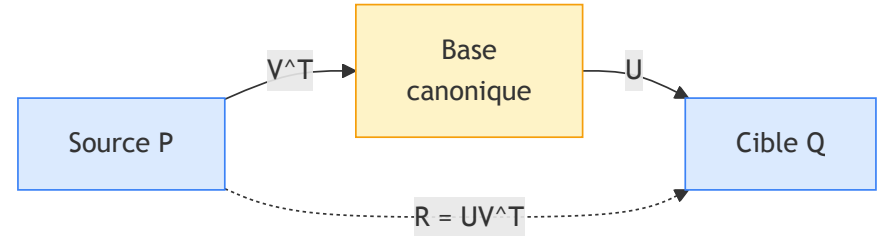
1. Ce que contient W ($Q'P'^T$)

Cette matrice capture comment les axes du nuage P sont corrélés avec les axes du nuage Q .

2. Le rôle de la SVD (UDV^T)

La SVD décompose toute transformation linéaire en :

- V^T : Rotation (aligne P sur une base canonique).
- D : Étirement (scaling).
- U : Rotation (projette vers la base de Q).



Comme on cherche une transformation **rigide** (pas de déformation), on ignore l'étirement D :

$$R = U \cdot \underbrace{I}_{\text{ignore } D} \cdot V^T = UV^T$$

Résolution par SVD : Translation

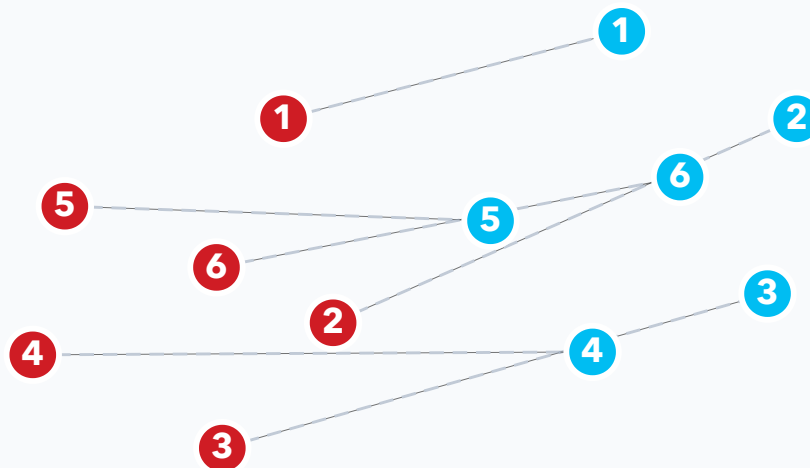
Une fois la rotation optimale R connue, la translation est la différence entre les centres de masse, corrigée par la rotation.

$$t = \mu_Q - R \mu_P$$

Résumé de l'étape d'estimation

L'approche SVD donne la solution optimale au sens des moindres carrés pour des correspondances fixes, sans itération interne.

Démonstration interactive : Estimation par SVD



Source P (rouge), cible Q (bleu) – correspondances $1 \leftrightarrow 1, 2 \leftrightarrow 2, \dots$

● Source P ● Cible Q 1 / 6

◀ Précédent

Suivant ▶

▶ Auto

↺ Reset

Variante importante : Point-to-Plane

La métrique "Point-to-Point" converge lentement dans les environnements avec des ambiguïtés géométriques (ex : couloirs).

Point-to-Point :

$$\|q - (Rp + t)\|^2$$

Minimise la distance euclidienne directe.

Point-to-Plane :

$$\|n_q \cdot (q - (Rp + t))\|^2$$

Minimise la distance projetée sur la normale n_q de la surface cible.

- **Avantage** : Permet au nuage de "glisser" le long des surfaces planes, convergence plus rapide et meilleure précision géométrique.
- **Requis** : Il faut calculer les normales de surface.

Gestion des Outliers (Données aberrantes)

L'ICP naïf est très sensible aux outliers (moindres carrés quadratiques).

Stratégies de rejet de correspondances :

1. **Seuil de distance max** : Si $\|p_i - q_j\| > d_{\max}$, on ignore la paire.
2. **Compatibilité des normales** : Si l'angle entre les normales n_p et n_q est trop grand ($> 45^\circ$), ce n'est pas la même surface.
3. **Trimmed ICP** : On trie les erreurs et on ne garde que les $X\%$ meilleures (ex : 90%) pour calculer la transformation.

Limitations de l'ICP

Problème de convergence locale

- Il converge vers le minimum local le plus proche.
- **Conséquence** : Il nécessite une bonne initialisation (ex : odométrie, vitesse constante).
- **TP 1**: Vous verrez comment associer des points caractéristiques permet de résoudre ce problème.

Applications typiques :

- Recalage scan-à-scan (Odométrie LiDAR).
- Recalage scan-à-carte (Localisation).
- Suivi d'objets (Object Tracking).